Assignment 10

This exercise has two parts:

- Part A consists of mandatory exercises; you need to complete these and hand in your answers to be able to partake in the exam. This part also provides you with an impression of the minimum knowledge level necessary to pass the exam.
- Part B consists of additional exercises. These repeat a concept that is covered in Part A or help to broaden or deepen your understanding of some of the topics.

Assignment by

NAMES AND STUDENT NUMBERS

Advised Reading and Exercise Material

The following reading material is recommended:

Read and study chapter 5 of TSK. You can skip section 5.2.6.

Additional Tools

For this exercise you will need to load the provided *clusterPlot.py* and *clusterVal.py* scripts from the toolbox folder.

Introduction

In previous exercises we considered supervised learning, i.e., we were given both input data **X** and output values **Y**. We now move on to unsupervised learning where we are only provided input data **X**. The aim is here to find common patterns in the data such as groups of observations that are similar in some sense.

Part A - Mandatory Exercises

Below you will find the mandatory exercises for Week 4 of the course Data Mining and Machine Learning.

Exercise 1 - *k*-means clustering

In this part of the exercise we will investigate *k*-means clustering. In *k*-means each of the data points are assigned to the cluster in closest proximity according to some measure of distance between cluster centers and data points. When the distance is given by the squared euclidian distance, the centers are also called centroids. Once the data points have been assigned, each cluster center is updated to be placed at the center of the data points that are assigned to the cluster. This continues iteratively, usually until the assignment of data points to centers no longer changes or until a maximal number of iterations is reached.

Exercise 1.1

Load the `synth1` - `synth4` datasets into python using the `loadmat` function. The `synthX` data is a Matfile loaded into Python as a dictionary; the relevant data is contained under keys `X` and `y`.

In [10]:

```
## Answer to exercise 1.1
import numpy as np
import scipy.io as si


X1 = si.loadmat('data/synth1.mat')['X']      #n x 2 matrix of points in synth1
y1 = si.loadmat('data/synth1.mat')['y']      #true class labels in synth1
X2 = si.loadmat('data/synth2.mat')['X']
y2 = si.loadmat('data/synth2.mat')['y']
X3 = si.loadmat('data/synth3.mat')['X']
y3 = si.loadmat('data/synth3.mat')['y']
X4 = si.loadmat('data/synth4.mat')['X']
y4 = si.loadmat('data/synth4.mat')['y']

X = np.concatenate((X1, X2, X3, X4))        #if all the data is wanted in the next exercise than this is how we can concatenate data from all files
y = np.concatenate((y1, y2, y3, y4))
```

Exercise 1.2

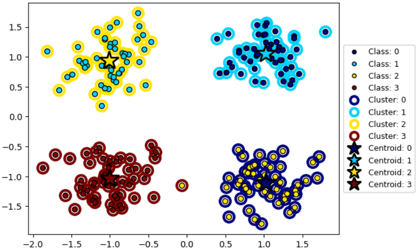
Cluster the data into `K = 4` clusters using the *k*-means algorithm with the euclidian distance metric. Make a scatter plot of the data and the clustering using the provided `clusterPlot()` function from the toolbox. Move the legend if the position is obfuscating data.

Helpful hints:

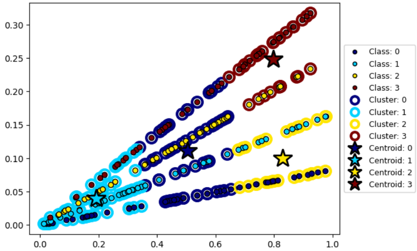
- Type `clusterPlot(X,clusters,centroids,y)` to plot the data and the clustering. Don't forget to import the function from the `toolbox` folder. If you are unsure about how to use the function, open the .py file in a text editor and take a look at documentation.
- In Python, you can use the function `KMeansClusterer()` from the package nltk.cluster.kmeans to compute *k*-means clustering. please read the documentation of the function, as it creates an object which you can then use for clustering.
- Use distance = nltk.cluster.util.euclidean_distance to use the euclidean distance metric!

In [19]:

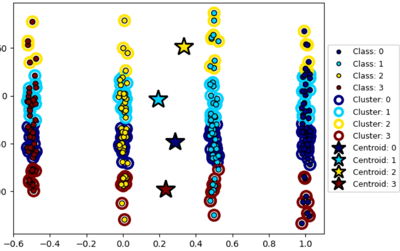
```
## Answer to exercise 1.2
from toolbox import clusterPlot as cp
from nltk.cluster.kmeans import KMeansClusterer
from nltk.cluster.util import euclidean_distance
k1 = 4

def clustering(k, x, Y, metric):
#first we need to get clusters and centroids:
    clusterer = KMeansClusterer(k, metric, avoid_empty_clusters=True)
    clusters = clusterer.cluster(x, True)  #getting clusters from the clusterer
    centroids = clusterer.means()       #and getting the means AKA centroids from the clusterer

    #now we have everything we need for clusterPlot:
    cp.clusterPlot(x, clusters, centroids, Y)

clustering(k1, X1, y1, euclidean_distance)
```



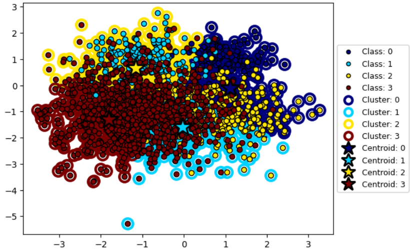
```
In [12]: clustering(k1, X2, y2, euclidean_distance)
```



```
In [4]: clustering(k1, X3, y3, euclidean_distance)
```



```
In [13]: clustering(k1, X4, y4, euclidean_distance)
```



Exercise 1.3

Does the clustering coincide with the true class? Explain for each of the four datasets.

ANSWER:

The clustering of the synth1 **does coincide** with the true class since from the clusterplot we can clearly see (and also from assignment 9) that we have 4 distinct clusters, which is also confirmed by the legend. All but one points are classified correctly.

The clustering of the synth2 **does not concide** well with the true class since from the clusterplot we can see that there are indeed 4 class labels but the clustering was unsuccessful (or very wrong). The clusters were also swayed by the outliers.

The clustering of synth3 **does not concide** well with the true class since from the clusterplot we can see that there are 4 class labels but the clustering was again unsuccessful. This happened because the objects aren't spherical. A DBSCAN would work well here.

The clustering of synth4 **does not concide** well with the true class since from the clusterplot we can see that the data is probably uncorrelated. So we're searching for a pattern that isn't there.

Evaluating how good your clustering is can be hard, because we don't always have labels. Generally, clustering is **unsupervised**. If we do however have labels, we can use the supervised measures of cluster validity described in *Introduction to Data Mining* section 8.5.7, in particular **the entropy, purity, rand statistic, and Jaccard coefficient**. Carefully review these measures in the book and make sure you understand how they are calculated.

Exercise 1.4

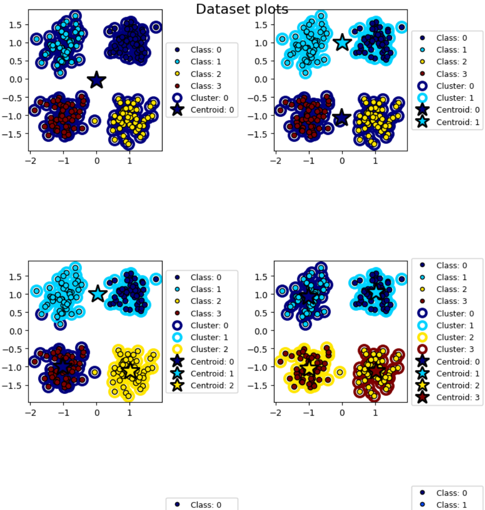
Perform `k` -means clustering for $k = 1, \ldots, 10$ clusters on each of the datasets. For each value of `k` compute the four cluster validity measures mentioned above (you don't need to implement them yourselves, see hints below!!). For each dataset: plot the calculated cluster validity measures as a function of `k`. Add a legend to each figure in order to indicate which measures are plotted in which colour.

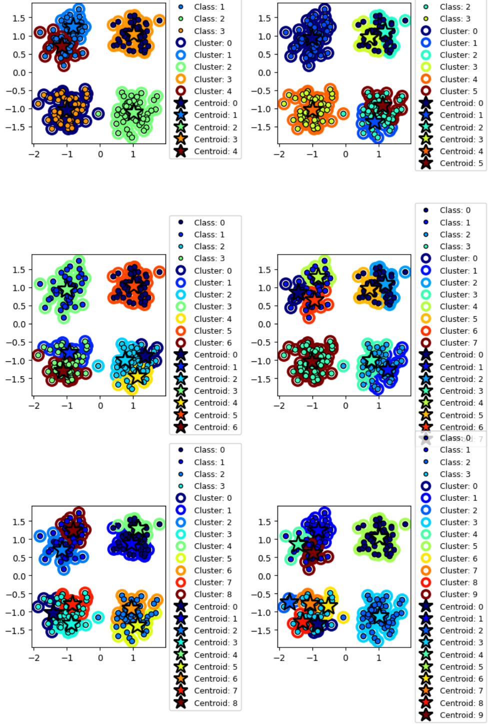
Helpful hints:

- Use the `clusterVal` function to compute the cluster validity. Don't forget to import it from the `toolbox folder`. If you are unsure about how to use the function, open the .py file in a text editor and take a look at documentation.
- Labels for a legend are most easily generated by calling `plt.plot` multiple times before calling `plt.show`. You can set the `label` argument in each `plt.plot` to the name of the label (e.g. what you're plotting). Read the documentation in case you are uncertain!

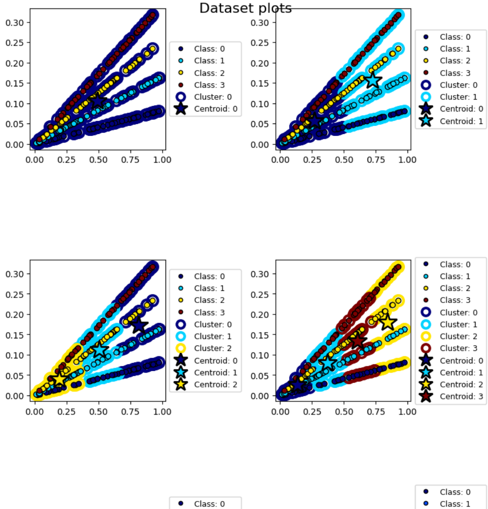
In [14]:

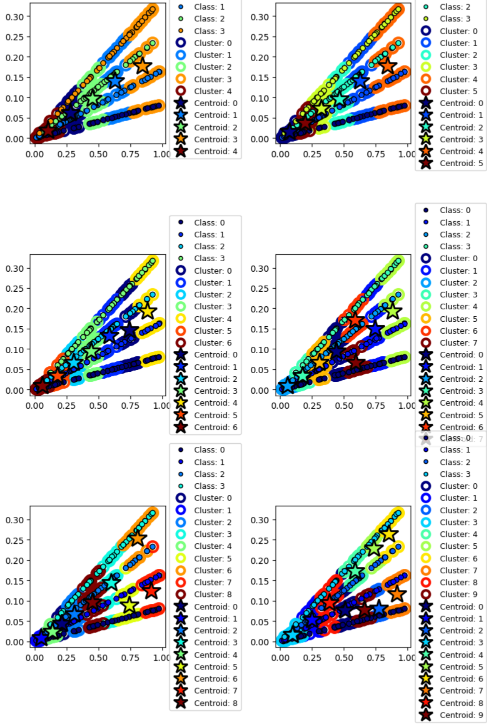
```
## Answer to exercise 1.3
from toolbox import clusterVal as cv
import matplotlib.pyplot as plt

#plotting all the clusterings for better visualization (we'll get 4 * 10 plots because there are 4 datasets for which we plot 10 plots; one for each k):

#This version is without the use of subplots for easier understanding:
""" for x, y in [(X1, y1), (X2, y2), (X3, y3), (X4, y4)]:
    for k in range(1, 11):
        plt.figure()  #we need to specifically say to create a new figure each time
        clustering(k, x, y)
        plt.show() """

#I want to revise making subplots so:
def kmeans_subplots(K):
    for x, y in [(X1, y1), (X2, y2), (X3, y3), (X4, y4)]:
        fig, axs = plt.subplots(5, 2, figsize=(8, 20))   #we have 10 plots for each dataset and we'll separate them into 5 rows of 2
        fig.suptitle("Dataset plots", fontsize=16)

        for idx, k in enumerate(range(1, K + 1)):
            ax = axs[idx // 2][idx % 2]     #picking position in the 5 x 2 grid
            plt.sca(ax)     #settinf the current axis
            clustering(k, x, y, euclidean_distance)

        plt.tight_layout()
        plt.show()



kmeans_subplots(10)
```



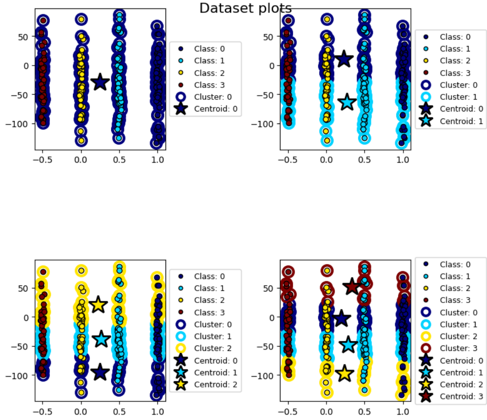


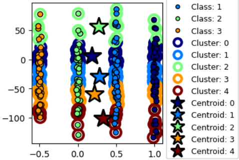
Dataset plots

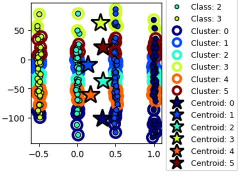


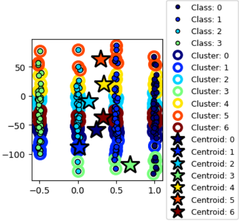


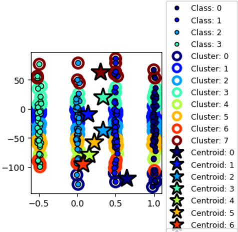
Dataset plots

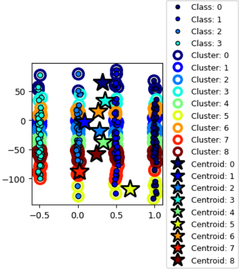


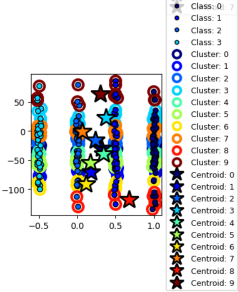




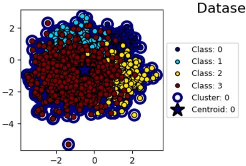


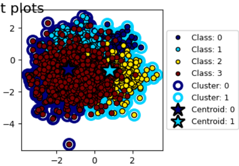


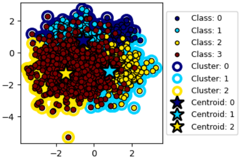


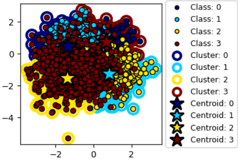


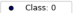
Dataset plots

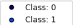


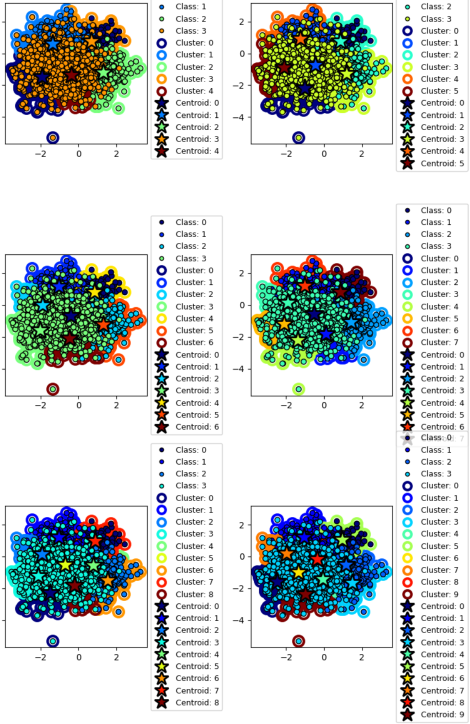






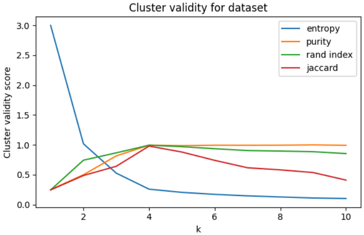


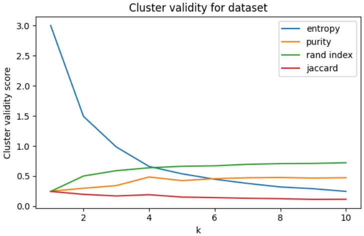


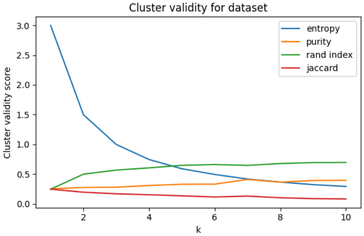


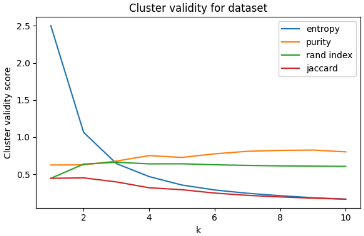
In [34]:

```
def evaluation_of(K, metric):
    for x, y in [(X1, y1), (X2, y2), (X3, y3), (X4, y4)]:
        entropies  = []     #initiating all the evaluators
        purities   = []
        rands      = []
        jaccards   = []

        for k in range(1, K + 1):
            clusterer = KMeansClusterer(k, metric, avoid_empty_clusters=True)
            clusters = clusterer.cluster(x, True)

            entropy, purity, rand, jaccard = cv.clusterVal(y, clusters)     #calculating all validity measures

            entropies.append(entropy)
            purities.append(purity)
            rands.append(rand)
            jaccards.append(jaccard)

        #plotting
        plt.figure(figsize=(6, 4))
        ks = list(range(1, K + 1))

        plt.plot(ks, entropies, label="entropy")
        plt.plot(ks, purities, label="purity")
        plt.plot(ks, rands, label="rand index")
        plt.plot(ks, jaccards, label="jaccard")

        plt.xlabel("k")
        plt.ylabel("Cluster validity score")
        plt.title("Cluster validity for dataset")
        plt.legend()
        plt.tight_layout()
        plt.show()


evaluation_of(10, euclidean_distance)
```









Exercise 1.5

How can the cluster validity measures be used to select the best number of clusters? Does one of the datasets have a clear optimum?

> What happens when more than four clusters are used to model the data? Explain for each dataset.

ANSWER:

Entropy can indicate the best number of clusters k by where the knee of the plot is. For the first dataset that would be either 2, 3 or 5. Purity generally increases as we increase the number of clusters. If the plot of purity agains k has some obvious "peaks" then we can try using that k as the number of clusters. Rand index works symmilarly. If we look at jaccard coefficient plotted against k, we can observe if we have any extreme maximum values. Those indicate the best choices for k determined by the jaccard. None of the k's in the clusters 2, 3 and 4 stand out as the best, but in dataset 1 we can observe that for K = 2, 3, 5 most of the measures indicate a good choice for k. This is interesting since you would think that there should be 4 clusters by looking at the plot of clusters. This just means that the measures only suggest good k's and don't produce the perfect one even if they all indicate one value for k.

After running the algorithm 2 more times the best value for k in the first dataset "changed" to 4. So by calculating the measures more than once we can get different results. If the results fluctuare a lot, the clustering is less stable.

Exercise 2 - Different distance metrics

Exercise 2.1

For each dataset answer the following questions:

- Does the current *k*-means clustering work well? (see the validity at the end of 1)
- Do you think that either a weighted version of euclidean (x>y or vice versa) or the cosine distance would be better?

ANSWER:

For the first dataset, the k-means clustering works well. We can already see this from plotting the kmeans clustering, but if we only focus on the validity measures, we can see that for example purity rises to almost 1 at k = 5 and never decreases (or after 4 in the second try).

For the other 3 datasets the current k-means clustering does not work well. This is because the clusters aren't globular, which can be seen from the kmeans plot. The purity measure of each of them moves around 0.5 which is very average (random).

I don't know which distance would work better.

Exercise 2.2

Cluster each dataset again, but use a different distance metric where you think it would help!

Helpful hints:

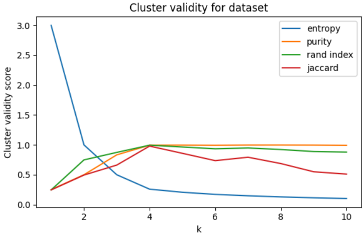
- Use the implemented weighted_euclidean or nltk.cluster.util.cosine_distance as the distance

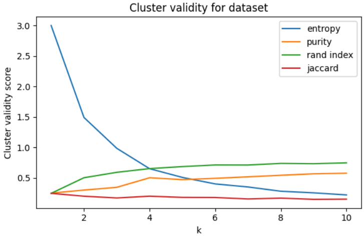
In [63]:

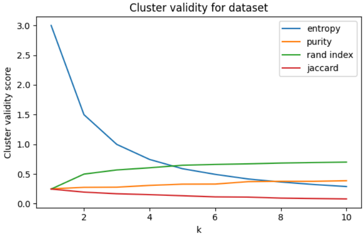
```
import numpy as np
from nltk.cluster.util import cosine_distance
weights = np.array([0.3,0.7])  # adjust as needed

def weighted_euclidean(u, v):
    u = np.asarray(u)
    v = np.asarray(v)
    return np.sqrt(((weights * (u - v)) ** 2).sum())
```

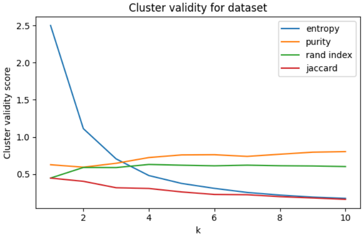
In [64]:

```
#first let's try the weighted euclidean:
evaluation_of(10, weighted_euclidean)
```

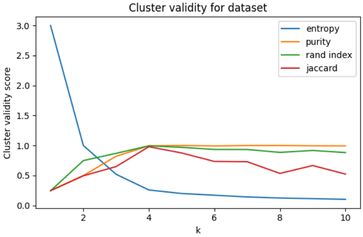


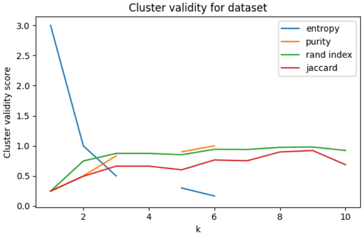


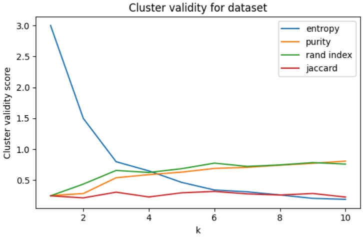


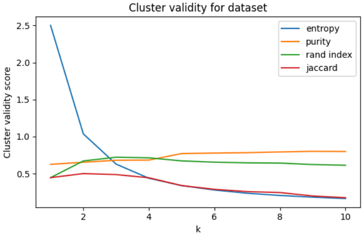


```
In [51]: evaluation_of(10, cosine_distance)
```









Exercise 2.3

What are your final distance measures used for each cluster?

ANSWER:

for the first dataset, we can just use the normal euclidean distance we used in exercise 1, since it already told us which values to try for k.

For the second dataset, we can use the cosine metric since all of the measures tell us that a good choice for k is either 4 or 5, which is a close to optimal choice even though we cannot get the correct clustering from kmeans clustering.

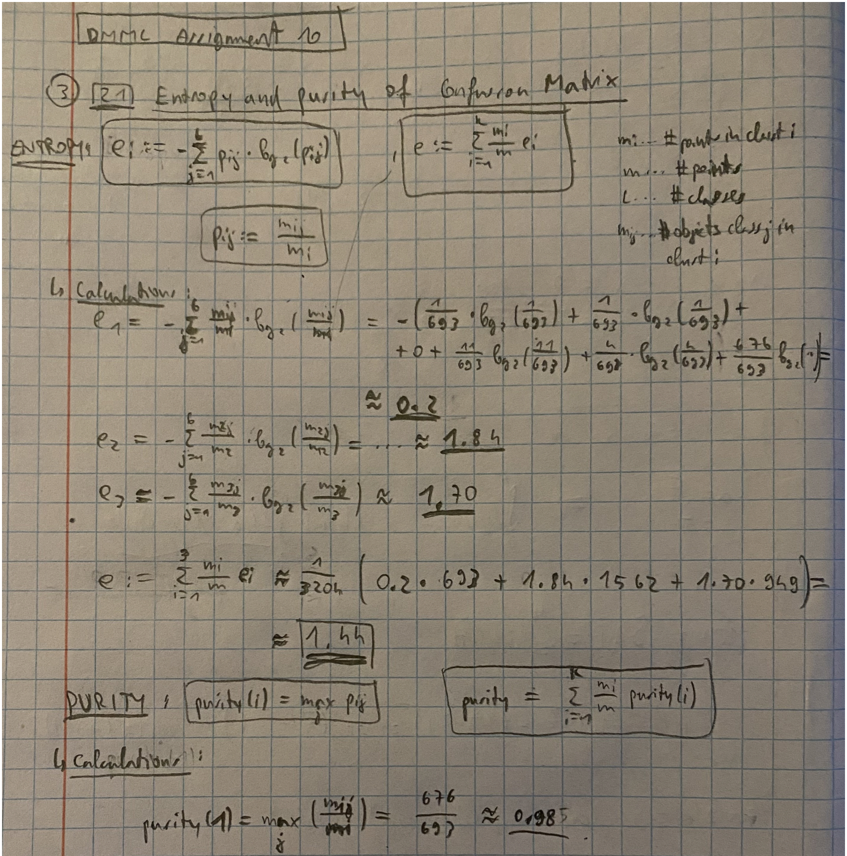
For the third dataset, it's very hard to get a good k for the clustering. The weighted euclidean suggested k = 3 most times, which is close to the optimal value of clusters.

For the fourth dataset, all of the measures still differ and fluctuate a lot, meaning that we don't have a particulat pattern.

Exercise 3 - Validity Measures on paper

1. Do exercise 21 from section 5.7 in TSKK.
2. Do exercise 24 from section 5.7 in TSKK.

If you do it on paper, including one (1) scan of all your on paper work (as a PDF) next to your main PDF/ipynb file is perfectly fine!



DMML Assignment 10

③ 21 Entropy and purity of Confusion Matrix

ENTROPY: $\boxed{e_i := -\sum_{j=1}^{L} p_{ij} \cdot \log_2(p_{ij})}$, $\boxed{e := \sum_{i=1}^{k} \frac{m_i}{m} e_i}$

m_i … # points in clust i

m … # points

L … # classes

m_{ij} … # objects class j in clust i

$\boxed{p_{ij} := \frac{m_{ij}}{m_i}}$

↳ Calculations:

$$e_1 = -\sum_{j=1}^{6} \frac{m_{1j}}{m_1} \cdot \log_2\left(\frac{m_{1j}}{m_1}\right) = -\left(\frac{1}{693} \cdot \log_2\left(\frac{1}{693}\right) + \frac{1}{693} \cdot \log_2\left(\frac{1}{693}\right) + 0 + \frac{11}{693} \log_2\left(\frac{11}{693}\right) + \frac{4}{693} \cdot \log_2\left(\frac{4}{693}\right) + \frac{676}{693} \log_2(\cdot)\right) \approx \underline{0.2}$$

$$e_2 = -\sum_{j=1}^{6} \frac{m_{2j}}{m_2} \cdot \log_2\left(\frac{m_{2j}}{m_2}\right) = \ldots \approx \underline{1.84}$$

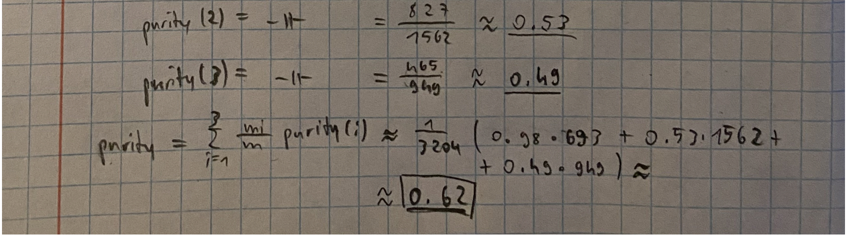
$$e_3 \approx -\sum_{j=1}^{6} \frac{m_{3j}}{m_3} \cdot \log_2\left(\frac{m_{3j}}{m_3}\right) \approx \underline{1,70}$$

$$e := \sum_{i=1}^{3} \frac{m_i}{m} e_i \approx \frac{1}{3204}\left(0.2 \cdot 693 + 1.84 \cdot 1562 + 1.70 \cdot 949\right) = \approx \boxed{\underline{1,44}}$$

PURITY: $\boxed{\text{purity}(i) = \max_j p_{ij}}$ $\boxed{\text{purity} = \sum_{i=1}^{K} \frac{m_i}{m} \text{purity}(i)}$

↳ Calculations:

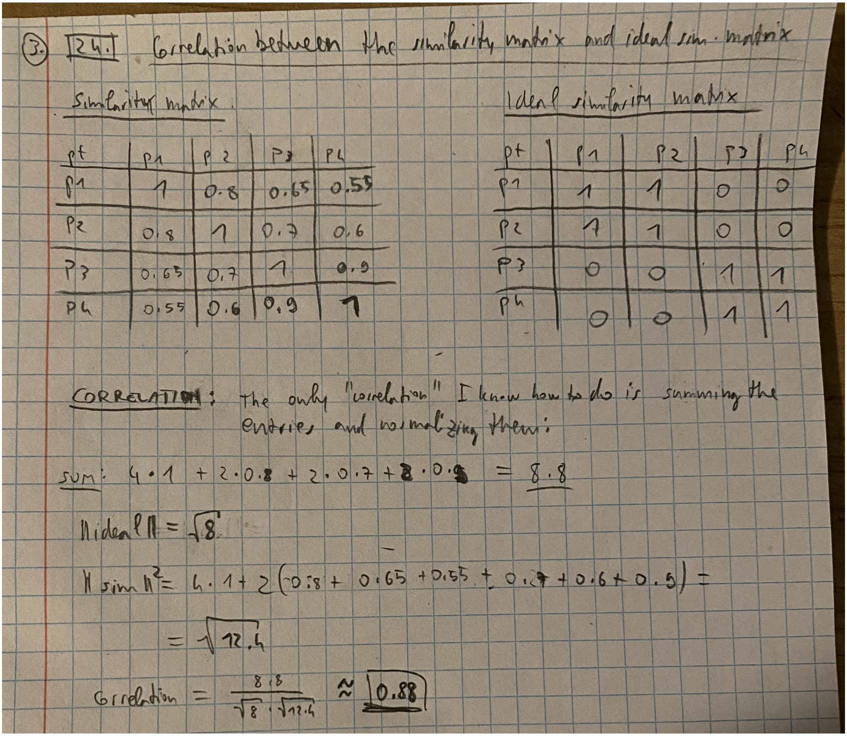
$$\text{purity}(1) = \max_j \left(\frac{m_{1j}}{m_1}\right) = \frac{676}{693} \approx \underline{0,985}$$



$\text{purity}(2) = \text{-}\|\text{-} = \frac{827}{1562} \approx \underline{0.53}$

$\text{purity}(3) = \text{-}\|\text{-} = \frac{465}{949} \approx \underline{0.49}$

$$\text{purity} = \sum_{i=1}^{3} \frac{m_i}{m}\,\text{purity}(i) \approx \frac{1}{3204}\left(0.98\cdot 693 + 0.53\cdot 1562 + 0.49\cdot 949\right) \approx \boxed{0.62}$$



③ $\boxed{2\text{4.}}$ Correlation between the similarity matrix and ideal sim. matrix

Similarity matrix

pt	p1	p2	p3	p4
p1	1	0.8	0.65	0.55
p2	0.8	1	0.7	0.6
p3	0.65	0.7	1	0.9
p4	0.55	0.6	0.9	1

Ideal similarity matrix

pt	p1	p2	p3	p4
p1	1	1	0	0
p2	1	1	0	0
p3	0	0	1	1
p4	0	0	1	1

CORRELATION: The only "correlation" I know how to do is summing the entries and normalizing them:

SUM: $4\cdot 1 + 2\cdot 0.8 + 2\cdot 0.7 + 2\cdot 0.9 = \underline{8.8}$

$\|\text{ideal}\| = \sqrt{8}$

$\|\text{sim}\|^2 = 4\cdot 1 + 2\left(0.8 + 0.65 + 0.55 + 0.7 + 0.6 + 0.9\right) =$

$= \sqrt{12.4}$

$$\text{Correlation} = \frac{8.8}{\sqrt{8}\cdot\sqrt{12.4}} \approx \boxed{0.88}$$

Exercise 4 - DBScan

In this part of the exercise we will investigate DBSCAN clustering. In DBSCAN, clusters are formed by identifying regions of high point density rather than assigning points to the nearest center. A point is considered a core point if it has enough neighboring points within a specified radius, and clusters are created by expanding outward from these core points to all other points that are reachable through dense regions. Points that do not belong to any such region are labeled as noise. Unlike k-means, DBSCAN does not iteratively update cluster centers, nor does it require the number of clusters to be specified in advance; instead, clustering finishes once all points have been visited and all dense regions have been fully expanded.

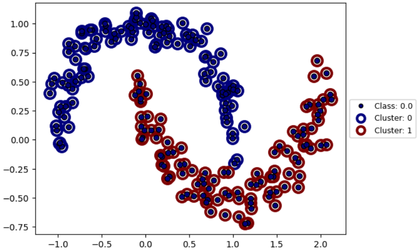
Exercise 4.1

For DBScan we are going to work with the moons dataset. This is a very common dataset in clustering. We will use several instances of this dataset, which you will generate yourself.

- Use the make_moons function from sklearn.datasets to generate a dataset with 200 samples, with noise equal to 0.1.
- Plot the dataset with the colour corresponding to the labels.

In [ ]:

```
## Answer to exercise 3.1
from sklearn.datasets import make_moons
samples = 200

Xs, ys = make_moons(samples, noise=0.1)
cp.clusterPlot(Xs, ys)
```

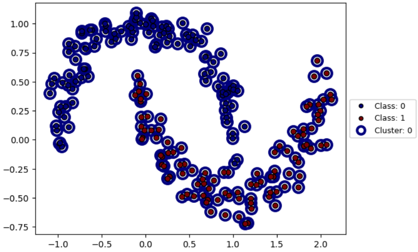


Exercise 4.2

Use the DBSCAN function from sklearn.cluster to fit a clustering to the dataset you loaded above. Pick eps and min_samples such that you find a reasonable clustering. Describe if it is easy/hard/impossible to cluster the data perfectly.

In [ ]:

```
## Answer to exercise 3.2
from sklearn.cluster import DBSCAN

clustering1 = DBSCAN(eps=0.2, min_samples=4).fit(Xs)     #and getting the means AKA centroids from the clusterer
labels = clustering1.labels_

#now we have everything we need for clusterPlot:
cp.clusterPlot(Xs, labels, y=ys)
#cp.clusterPlot(Xs, clustering1, ys)
```



YOUR ANSWERS:

It is impossible to cluster the data perfectly because of noise, but if we remove it, we can cluster it pretty easily and almost perfectly. Maybe not the best but good enough parameters are Eps = 0.2 and MinPts = 4. When we choose these parameters only 2 points are clustered incorrectly. Point (0.25, 1.25) can easily be reassigned to the right class (which is the closest) if we say that there have to be exactly 2 clusters. The other point that is incorrectly classified could not be assigned to the right class, since the second and third closest points to it are from the wrong class. So for MinEps >= 3 it will always be assigned to the wrong one.

Exercise 4.3

Now repeat 4.1 and 4.2 for several different parameters:

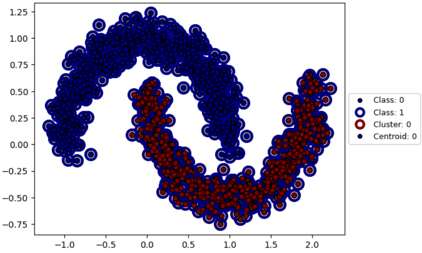
- a dataset with 1000 samples, with noise equal to 0.1
- a dataset with 100 samples, with noise equal to 0
- a dataset with 1000 samples, with noise equal to 0
- a dataset with 100 samples, with noise equal to .3

What do you notice?

- Do more samples always make it easier/harder?
- When is DBScan effective?

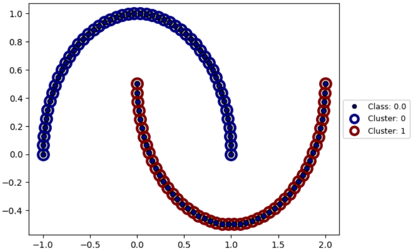
In [ ]:

```
## Answer to exercise 3.3

samples1000 = 1000
samples100 = 100
noise01 = 0.1
noise0 = 0
noise03 = 0.3

Xs1000, ys1000 = make_moons(samples1000, noise=0.1)
cp.clusterPlot(Xs1000, ys1000)

clustering2 = DBSCAN(eps=0.2, min_samples=13).fit(Xs1000)     #and getting the means AKA centroids from the clusterer
labels1 = clustering2.labels_

#now we have everything we need for clusterPlot:
cp.clusterPlot(Xs1000, labels1, y=ys1000)
```

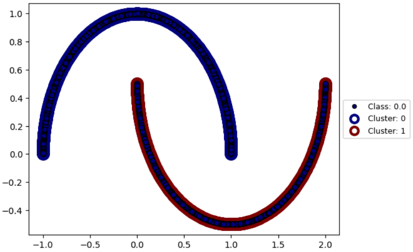


In this case when we only made the size of points larger, the DBSCAN can be more effective. It can classify the points perfectly.

In [ ]:
```
Xs100, ys100 = make_moons(samples100, noise=0)
cp.clusterPlot(Xs100, ys100)
```



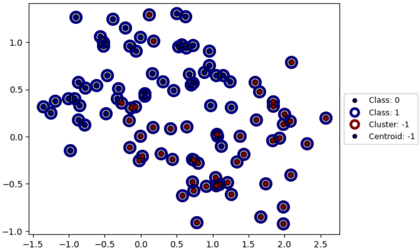
In [ ]:
```
Xs1000, ys1000 = make_moons(samples1000, noise=0)
cp.clusterPlot(Xs1000, ys1000)
```



ANSWER:

If the noise is 0, the DBSCAN can always make a perfect observation (given enough points). We can use a lot of different Eps and MinPts and still get the perfect clustering.

In [ ]:
```
Xs100, ys100 = make_moons(samples100, noise=0.3)
cp.clusterPlot(Xs100, ys100)

clustering3 = DBSCAN(eps=0.1, min_samples=5).fit(Xs100)     #and getting the means AKA centroids from the clusterer
labels2 = clustering3.labels_

cp.clusterPlot(Xs100, labels2, y=ys100)
```



ANSWER:

When the sample size is small and we have too much noise, the data cannot be distinguished and properly clustered with DBSCAN.

Part B

Exercise B.1

For further practice, e.g. to prepare for the exam: Do excercises 20, 22, 23, 37 in section 5.7 in TSKK.

Handing in

Hand in the filled-in notebook:

- Make sure your names and student-numbers are mentioned at the appropriate location at the top of this file.
- Make sure you wrote a serious attempt for all exercises in part A
- Make sure the notebook is named groupnumber_assignment10.ipynb

When you are confident all of this is correctly done, one of you can hand in the exercises in Brightspace!